

CSC 445 Assignment 2

Part 1: Primal vs. Dual LP Solving Efficiency

Joseph Bresee
V01005288

University of Victoria, Summer 2026

1 Introduction

The conjecture under investigation is:

“In practice, for a typical application that can be modeled by a linear program, it is more efficient to solve the dual LP than the primal LP to find the optimal solution.”

The purpose of this report is to investigate the above conjecture under the assumption that it literally means solving the dual LP with a solver, rather than using the dual simplex method or similar to solve the primal. This report aims to agree with or refute the conjecture of which formulation of a problem is more efficient to solve, which is to be distinguished from how algorithmic advantages can be found by exploiting duality theory while holding the formulation fixed.

When comparing a primal LP to its dual, a key fact must be considered: converting a primal LP to its dual swaps the roles of its m constraints and n variables, so that the dual has n constraints and m variables. This swap is for LPs in the form $Ax = b, x \geq 0$. It is hypothesised and tested in this report that simplex-based algorithms scale primarily with constraint count rather than variable count (Section 2), so the primal and dual efficiency of the same LP can differ substantially whenever $n \gg m$ or $n \ll m$, and how this information can be used to determine whether the primal or dual LP is more efficient to solve.

This report combines a review of established theoretical results on primal-dual computational asymmetry with an experimental comparison of primal and dual solve times, using real linear programs from the Netlib LP test set, an established set of LP tests used for benchmarking production linear program solvers. Additionally, a heuristic “rule of thumb” found online in a Q/A forum is tested (section 2.2) [2].

All code used for this investigation is available here:

<https://github.com/Joe-Bresee/linear-programming/tree/main/a2>

2 Theoretical Context

2.1 Duality and Computational Choice

Bradley, Hax, and Magnanti observe that when a problem’s constraint count greatly exceeds its variable count, solving the dual is usually preferable, since solve time grows faster with constraint count than with variable count [1].

This is not theoretical backing that the conjecture holds unconditionally, but a suggestion that the dual is *usually* the better option to solve under specific conditions. The experiment in Section 4 draws on Netlib instances whose column-to-row ratio spans from near-square up to roughly 421:1 (Table 1), to see whether there is evidence that the dual is “usually” better to solve as that ratio widens.

2.2 Complexity Asymmetry and LP-Solving Scale Hypothesis

A related heuristic, found in an online discussion of simplex vs. dual simplex methods, describes that LP-solving cost scales roughly with $rows^2 \times columns$, and recommends switching to whichever formulation has fewer rows once the transposed row count is meaningfully smaller [2].

Like the Bradley-Hax-Magnanti result, this source agrees that the dual may be easier to solve under specific circumstances, but it comes from a non-peer-reviewed practitioner discussion rather than a proven bound. It is included here as a hypothesis to test against the iteration counts collected in Section 5, not as an established fact. If this heuristic proves realistic, that is independent support beyond what Bradley, Hax, and Magnanti’s qualitative claim alone provides, backed by two separate sources and not just one.

2.3 Methodology, Practicality and the Chosen Solver

In this experimentation, HiGHS is used as the solver of choice due to its open-source nature and reputation for modern performance [3]. By using HiGHS, this report simulates practical, typical industrial linear program runs, reflecting the conjecture’s description of a “typical application.” For example, it would introduce artificial bias to test these formulations using SciPy’s now deprecated revised simplex method. Legacy implementations are known to be slow against equality-heavy constraints among other modern advantages and strategies internal to the HiGHS solver [5]. Using a solver like revised-simplex with more modern practical solver options available would taint the results measured by the algorithm due to its specific weaknesses rather than allow the efficiency differences of the primal versus dual formulations show in the runs. While the HiGHS solver exposed by Scipy also has its own known weaknesses, it is a currently supported, widely recognized production level solver appropriate to use in this experiment to appropriately test the conjecture’s statement.

HiGHS includes internal algorithmic optimizations, but also through SciPy, a solver-level setting can be tweaked to further investigate the formulations effect underneath the layers of solver engineering. There is a presolve feature with on/off options. testing with both presolve on and off, we can observe both the underlying capabilities of the solvers algorithm as well as any potential speedups the solver offers.

3 LP Model and Problem Formulation

To ensure the experiment measures performance on practical applications, instances were sourced from the LPnetlib collection of the SuiteSparse Matrix Collection [4]. This test set is derived from a real-world application of piecewise-linear data fitting in `fit1` and `fit2` (both `fit1` and `fit2` have primal (p) and dual(d) formulations available). Therefore, differences in solve times between primal and dual formulations seen from the experiment reflect real computational advantages/disadvantages driven by duality and matrix dimensions.

The LP instances are bounded-variable LPs of the form $Ax = b, 1 \leq x \leq u$ [4], where the variable bounds are handled separately from the rows of A instead of being additional constraint

rows. Therefore, the row and column counts shown later in Table 1 do not swap one-for-one as mentioned in the introduction. The row vs. columns asymmetry comparison still holds but is not the exact dimension swap between the primal and dual.

4 Experimental Setup

Each instance was solved with `scipy.optimize.linprog`, with HiGHS’s dual-simplex algorithm and HiGHS’s interior point method algorithm. Presolve was tested both on and off for every instance: HiGHS’s presolve runs a heuristic screening that may expose further exploitations the solver can take to speed up solve time. Both settings are tested to separate the primal/dual asymmetry effect on solve time from the solver-engineering contribution to solve time (Section 6). Each combination was solved 5 times, and the median of the 5 trials is taken. For every run, wall-clock solve time, simplex iteration count, solver status, and the solved objective value were recorded, and the solved objective was checked against each instance’s known-optimal value as a correctness check.

Both dual simplex and interior point methods were tested to determine whether the hypothesized primal-dual solve time asymmetry is limited to the simplex method’s vertex-hopping behaviour, or if it also shows in the interior point method, which would indicate a more fundamental mathematical property of linear programs. While the interior point method is typically understood to be more sensitive to problem density (Section 5), it may be useful to compare solve times and any seen effect on the method.

All instances were solved on a machine with Intel Core i7 with 16GB RAM running Windows.

Table 1: Matched primal/dual pairs from the LPnetlib collection [4]. Dimension ratios are in the form columns:rows ($n:m$).

Instance	Rows (m)	Columns (n)	Ratio ($n:m$)
<code>lp_fit1p</code> (Primal)	627	1,677	$\approx 2.7:1$
<code>lp_fit1d</code> (Dual)	24	1,049	$\approx 44:1$
<code>lp_fit2p</code> (Primal)	3,000	13,525	$\approx 4.5:1$
<code>lp_fit2d</code> (Dual)	25	10,524	$\approx 421:1$

`lp_fit1p/lp_fit1d` and `lp_fit2p/lp_fit2d` are known primal/dual pairs. Because the underlying problem is identical in each pair, these instances effectively isolate the formulation variable this experiment aims to show. Note that each run starts with the LP already in its desired form, so the small contribution to solve time that converting the LP to its primal/dual isn’t present.

5 Results

Figure 1 shows the median solve times for the two Netlib primal/dual pairs using HiGHS’s dual simplex backend (`highs-ds`), comparing performance with presolve enabled and disabled. Figure 2 shows the corresponding data using the interior point method (`highs-ipm`).

In dual simplex, solving the small row count formulations `fit1d` and `fit2d` shows definitive performance advantages over solving the large row count primals for these two LPs.

However, with the interior point method, the solve time difference between the primals and duals is nearly gone. This shows that HiGHS interior point method is not as sensitive to the dimensions

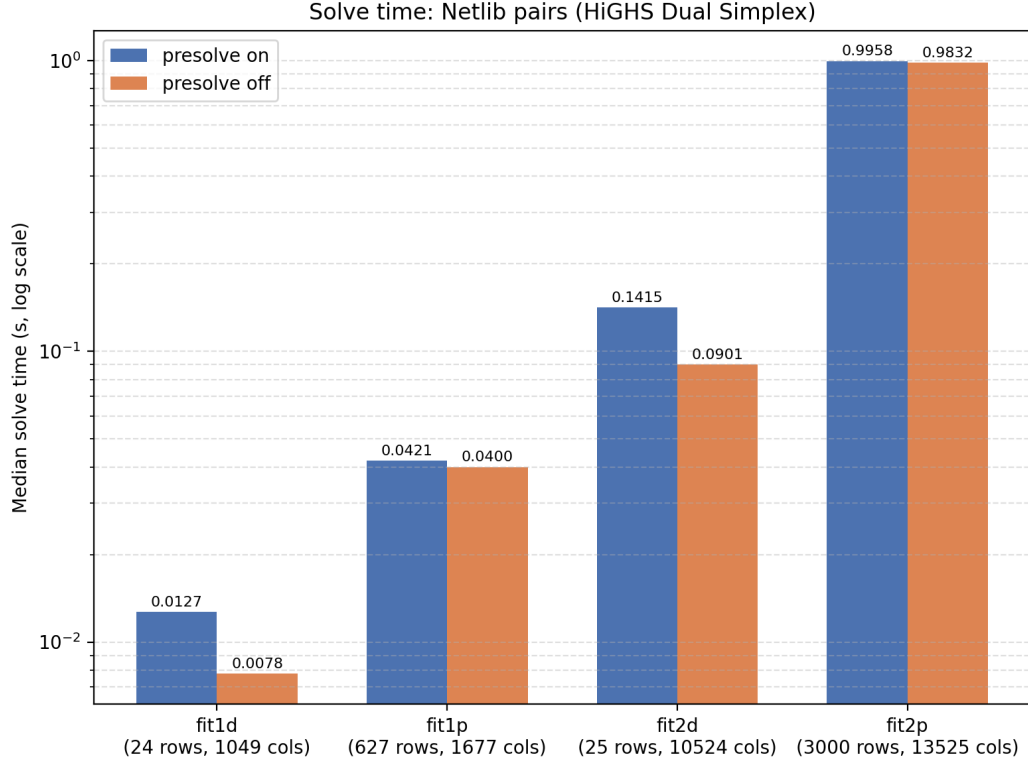


Figure 1: Median solve time (log scale), HiGHS dual simplex (**highs-ds**), for the two Netlib primal/dual pairs, presolve on vs. off. 5 trials per bar.

Table 2: Performance metrics for HiGHS dual simplex (**highs-ds**) across primal and dual formulations.

Instance	Rows (m)	Presolve	Iterations	Median Time (s)
lp_fit1d	24	On	69	0.0127
lp_fit1d	24	Off	75	0.0078
lp_fit1p	627	On	860	0.0421
lp_fit1p	627	Off	860	0.0400
lp_fit2d	25	On	118	0.1415
lp_fit2d	25	Off	136	0.0901
lp_fit2p	3000	On	4907	0.9958
lp_fit2p	3000	Off	4907	0.9832

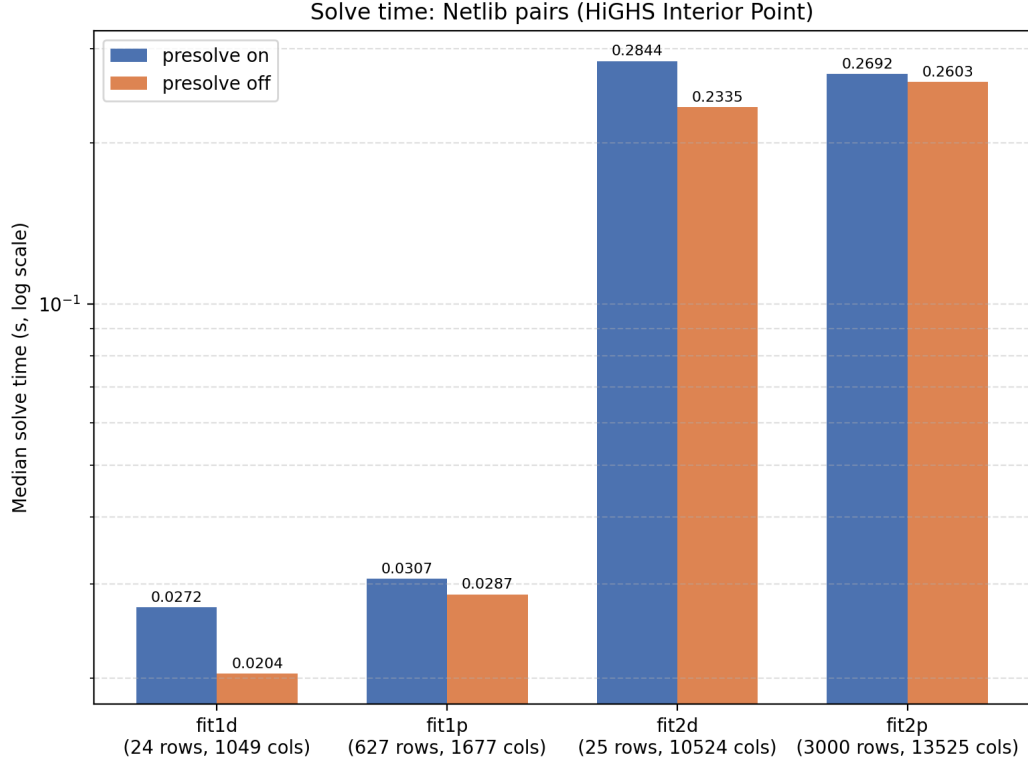


Figure 2: Median solve time (log scale), HiGHS interior point (**highs-ipm**), for the two Netlib primal/dual pairs, presolve on vs. off. 5 trials per bar.

Table 3: Performance metrics for HiGHS interior point (**highs-ipm**) across primal and dual formulations.

Instance	Rows (m)	Presolve	Iterations	Median Time (s)
lp_fit1d	24	On	18	0.0272
lp_fit1d	24	Off	18	0.0204
lp_fit1p	627	On	17	0.0307
lp_fit1p	627	Off	17	0.0287
lp_fit2d	25	On	23	0.2844
lp_fit2d	25	Off	22	0.2335
lp_fit2p	3000	On	20	0.2692
lp_fit2p	3000	Off	20	0.2603

asymmetry as HiGHS dual simplex is (interior point methods usually are more sensitive to the density of the problem).

Additionally, the interior point method run across presolve options appears to have: Slower solve times on `fit1.d`, faster solve times for `fit1.p`, slower solve times on `fit2.d`, and faster solve times on `fit2.p`.

Note the iteration count difference between dual simplex and interior point method. The interior point method has far less iterations than the dual simplex method. Since the problems from SuiteSparse [4], are sparse, it tracks with the known solve time density relationship of the interior point method.

6 Discussion

Regardless of presolve setting, the dual version of the given netlib LPs were solved faster than their primals for the dual simplex method. The presolve option seemed to slow down the solve time for the smaller matrices, while it had little impact on the larger matrix, `fit2p`. One guess is that these problems are too small for the presolve to have any meaningful positive effect on solve times, and therefore only introducing a small overhead cost in time to run the presolve before the rest of the algorithm. This bodes the same for the interior point method. The dual vs. primal solve times for the interior point method are similarly clustered, indicating little impact in solve time across the primal and dual. `fit1d` (24 rows) solved in roughly a third of `fit1p`'s (627 rows) time with presolve on, and roughly a sixth of its time with presolve off; `fit2d` (25 rows) solved in roughly an eighth of `fit2p`'s (3,000 rows) time with presolve on, and roughly a ninth of its time with presolve off. This does not follow the claim made in section 2.2, which stated the LP solving cost scales roughly with $rows^2 \times columns$, instead, it appears to be closer to $\sqrt{rows} \times columns$, which even with the smaller exponent demonstrates solve time dominance by row count. The member of each pair with far fewer rows was cheaper to solve, and the column count difference was much less between each pair's two members than the row count differed; `fit1d` has *fewer* columns than `fit1p` (1,049 vs. 1,677) and `fit2d` has fewer columns than `fit2p` (10,524 vs. 13,525), yet both dual-labelled instances still solved faster, consistent with row count rather than column count being the dominant driver here.

The results are representative of large-scale sparse optimization, which is the standard environment for practical industrial linear programming. Due to the speed of which interior point methods can solve problems in sparse problems, an LP-solving industry professional deciding which algorithm they should use in their approach would likely choose HiGHS-IPM from the 2 choices present.

7 Limitations

1. **Sample Size of Matched Pairs:** Only two matched primal/dual pairs exist in the sourced data (`fit1`, `fit2`). This is a very small set to test the conjecture on, however, in research online for appropriate primal dual pairs, this was the only set of pairs deemed acceptable to test with. A larger scoped investigation of this conjecture would have more test sets to experiment on which may find new/contrasting conclusions made by this one.
2. **Methodological Scope:** This is an experimental and theoretical investigation with an addition to a online-sourced heuristic, rather than a theoretical proof. The data obtained highlights practical behaviors on standard benchmarks, but does not actually make any attempts

to prove that the conjecture holds universally for all LP classes.

3. **Solver-Specific Scaling:** This experiment relies on HiGHS. While competitive on modest-sized LPs, HiGHS exhibits a significant performance gap on very large-scale instances. Benchmarks on massive energy system models demonstrate that HiGHS can be 60 to 100 times slower than commercial solvers like Gurobi, particularly when using interior point methods. Conclusions drawn here may be influenced by HiGHS’s specific implementation and engineering rather than representing a universal property of simplex or interior-point algorithms at massive scale.

8 Conclusion

The two Netlib primal/dual pairs support the conjecture, to an extent: in both `fit1p/fit1d` and `fit2p/fit2d`, the member with far fewer rows solved faster than its counterpart for the dual-simplex method, but not the interior point method, and this gap was consistent regardless of presolve settings. This suggests Bradley, Hax, and Magnanti’s claim (Section 2.1) - correctly identifies that row to column ratio does indeed drive solve times. Whichever formulation happens to have fewer rows is the one worth solving, and for these two models that formulation is the “dual.”

The heuristic from section 2.2 is directionally consistent, but is not a good rule of thumb for estimating computational complexity of solving an LP. Instead, what seems to be a more representative claim is that the computational complexity roughly scales with $\sqrt{\text{rows}} \times \text{columns}$. This heuristic is more consistent with the findings.

The interior-point method is not affected by the primal/dual asymmetry at all, but demonstrates speed on sparse problems. Because of this, it is likely the better option to choose the interior point method with problems like these; however in any case where this isn’t possible for whatever reason, the dual simplex method solving the problem with least rows is the best remaining choice.

Therefore, the conjecture that “In practice, for a typical application that can be modelled by a linear program, it is more efficient to solve the dual LP than the primal LP to find the optimal solution.”, is conditionally true under the tense that the problem is solved by a HiGHS dual-simplex solver, on a problem modelling piecewise-linear data fitting, with a high variable/constraint dimensional asymmetry. This may indicate the conjecture is conditionally true generally across simplex methods of various practical, typical applications modelled by linear programs, with high variable/constraint dimensional asymmetry.

References

- [1] S. Bradley, A. Hax, and T. Magnanti, *Applied Mathematical Programming*, Ch. 4: Duality in Linear Programming. [Online]. Available: <https://web.mit.edu/15.053/www/AMP-Chapter-04.pdf>
- [2] ResearchGate discussion, “How do I choose simplex method and dual simplex method in linear programming?” [Online]. Available: https://www.researchgate.net/post/How_do_I_choose_simplex_method_and_dual_simplex_method_in_linear_programming
- [3] HiGHS Documentation, “Solvers.” [Online]. Available: <https://ergo-code.github.io/HiGHS/dev/solvers/>; see also SciPy, “`linprog(method=‘highs-ds’)`” for the presolve option as exposed through the Python interface used in this report’s experiments. [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs-ds.html>

- [4] T. A. Davis and Y. Hu, “The University of Florida Sparse Matrix Collection,” *ACM Trans. Math. Softw.*, vol. 38, no. 1, Article 1, 2011. LPnetlib collection accessed via the SuiteSparse Matrix Collection. [Online]. Available: <https://sparse.tamu.edu>
- [5] SciPy Developers, “`scipy.optimize.linprog(method=‘revised simplex’)`” [Online]. Available: https://docs.scipy.org/doc/scipy/reference/optimize.linprog-revised_simplex.html